Asynchronous communications (AMQP)

The Advanced Message Queuing Protocol (AMQP) is an open standard designed to efficiently support a wide variety of messaging applications and communication patterns.

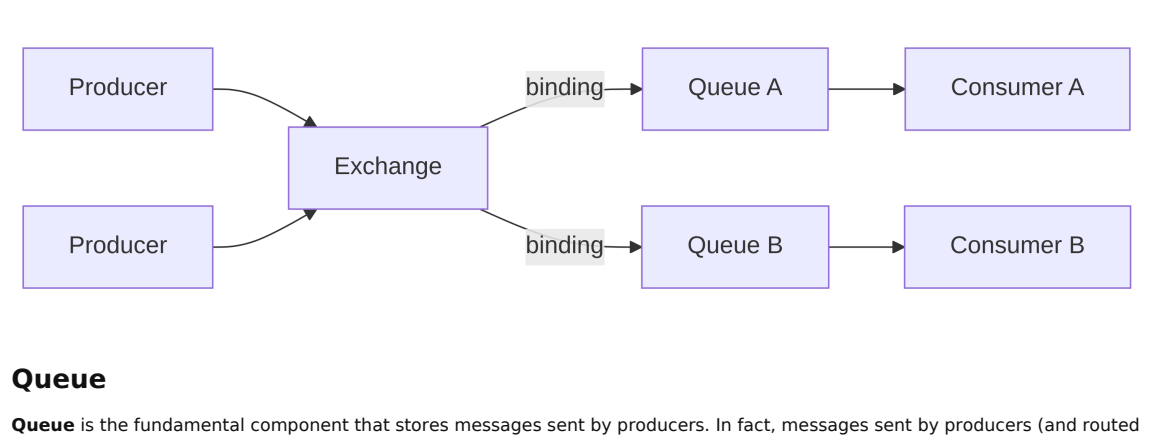
Broker	AMQP Protocol Version	Notes
RabbitMQ	AMQP 0-9-1	Open source, widely used, supports MQTT and STOMP. Great for microservices and enterprise systems.
LavinMQ	AMQP 0-9-1	Open source, written in Crystal, high-performance, compatible with standard AMQP clients.
Apache Qpid	AMQP 1.0	Apache project, offers broker and client implementations in Java, C++, Python. Suitable for enterprise systems.
ActiveMQ (Classic / Artemis)	AMQP 1.0	Supports STOMP, MQTT, OpenWire. Open source, flexible and robust.
Microsoft Azure Service Bus	AMQP 1.0	Cloud service for reliable messaging between distributed applications.
IBM MQ	AMQP 1.0	Enterprise broker, integrates with legacy systems and mainframes.
Red Hat AMQ	AMQP 1.0	Commercial distribution based on Apache Qpid/ActiveMQ. Enterprise-grade management and middleware integration.
Solace PubSub+	AMQP 1.0	Commercial broker optimized for real-time events and enterprise messaging; also supports MQTT and REST.

Key Components

- **Broker**: system that implements AMQP and handles message exchange between *producers* and *consumers*
- **Producer**: application that sends messages to broker
- **Consumer**: application that receives messages from broker

Broker is internally composed by two main components:

- **Exchange**
- **Queue**


Producer
Producer
Exchange
binding
binding
Queue A
Queue B
Consumer A
Consumer B

Queue

Queue is the fundamental component that stores messages sent by producers. In fact, messages sent by producers (and routed by exchanges) wait to be processed by consumer applications in queues.

- **Storage**: Queues store messages until they are processed or consumed by applications.
- **Message Order**: FIFO — First-In, First-Out.
- **Configurable Properties**: Queues have configurable properties such as maximum length, maximum priority levels, message TTL (Time-To-Live), etc., allowing fine-tuning to meet specific requirements.

Exchange

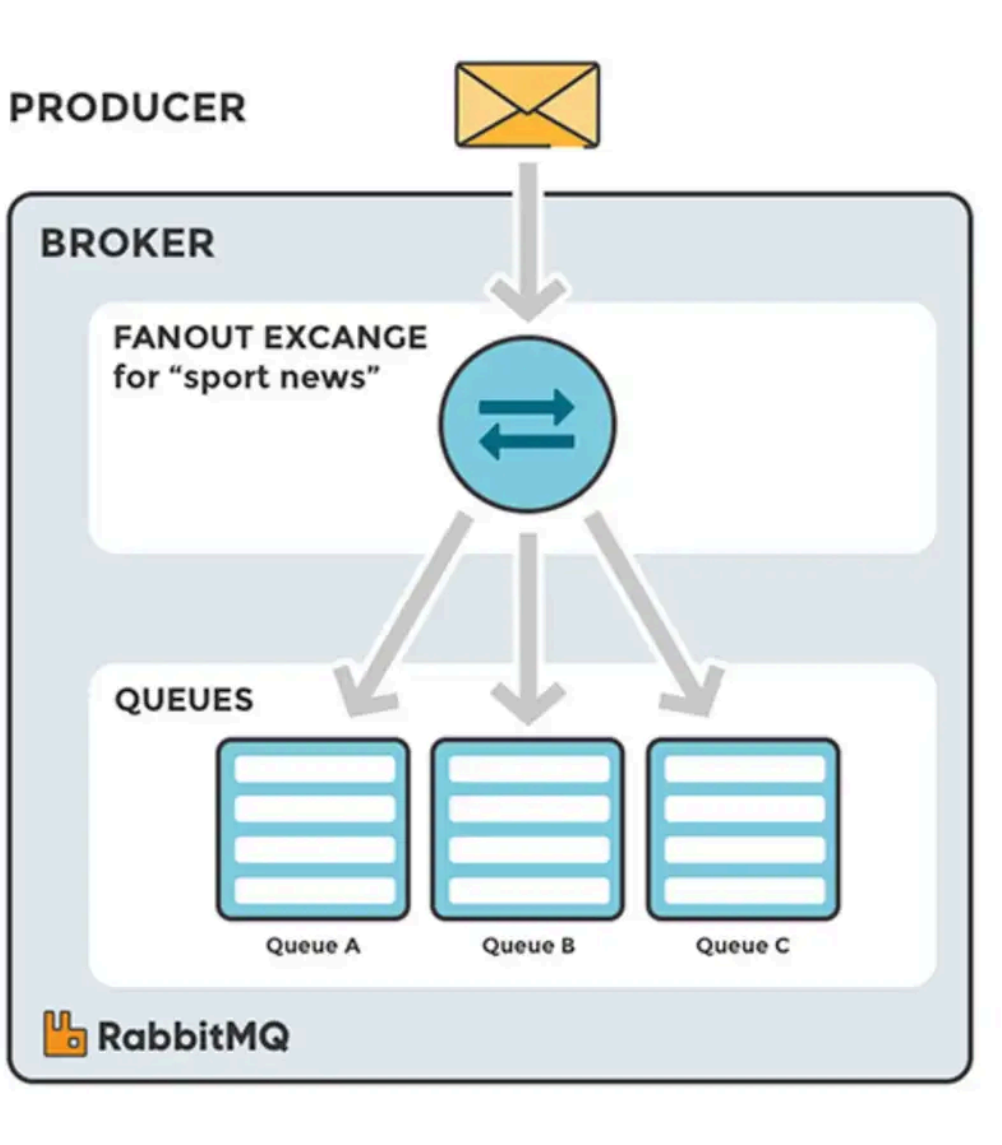
Exchange is the AMPQ entity which receives messages from producers and routes them to one or more queues based on routing rules.

The relationship between an exchange and a queue (including routing rules) is called **binding**.

- Fanout exchanges
- Direct exchanges
- Topic exchanges
- Header exchanges

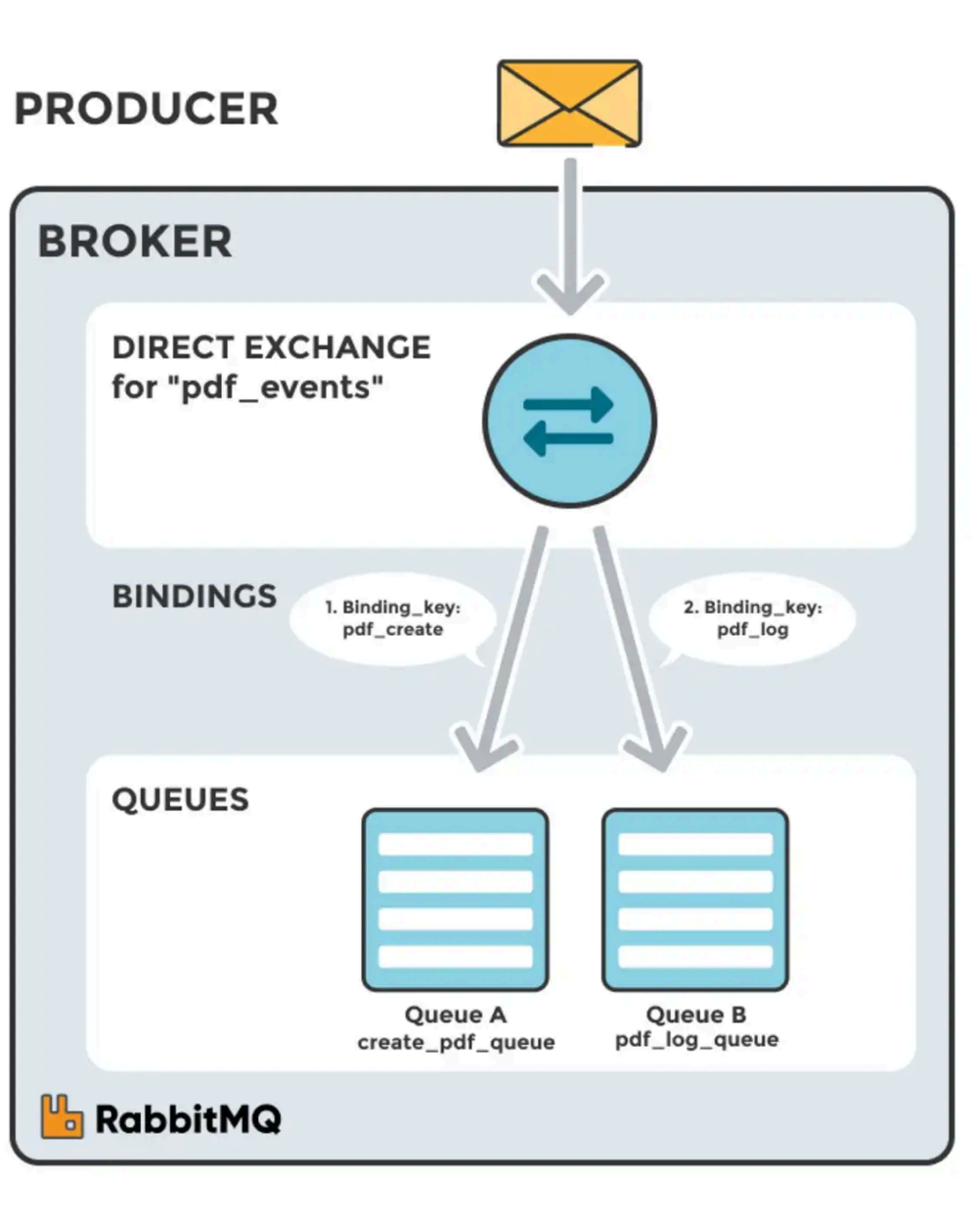
Fanout Exchange

A fanout exchange **routes messages to all queues that are bound to it, regardless of the routing key**. It broadcasts messages to multiple consumers.


PRODUCER
BROKER
FANOUT EXCANGE for "sport news"
QUEUES
Queue A
Queue B
Queue C
RabbitMQ

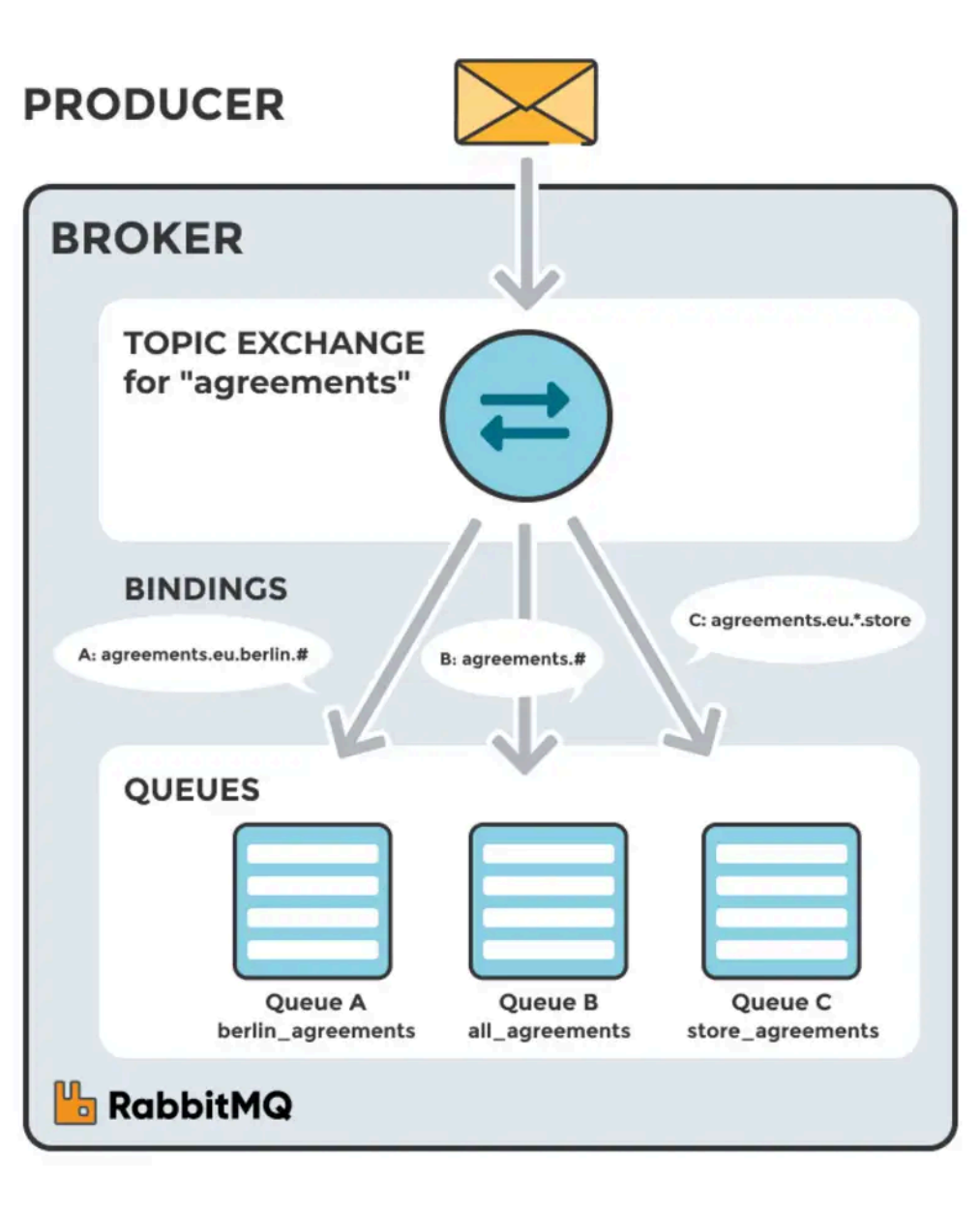
Direct Exchange

A direct exchange **routes messages with a specific routing key** to the queues that are bound to the exchange with the same routing key.


PRODUCER
BROKER
DIRECT EXCHANGE for "pdf_events"
BINDINGS
1. Binding_key: pdf_create
2. Binding_key: pdf_log
QUEUES
Queue A create_pdf_queue
Queue B pdf_log_queue
RabbitMQ

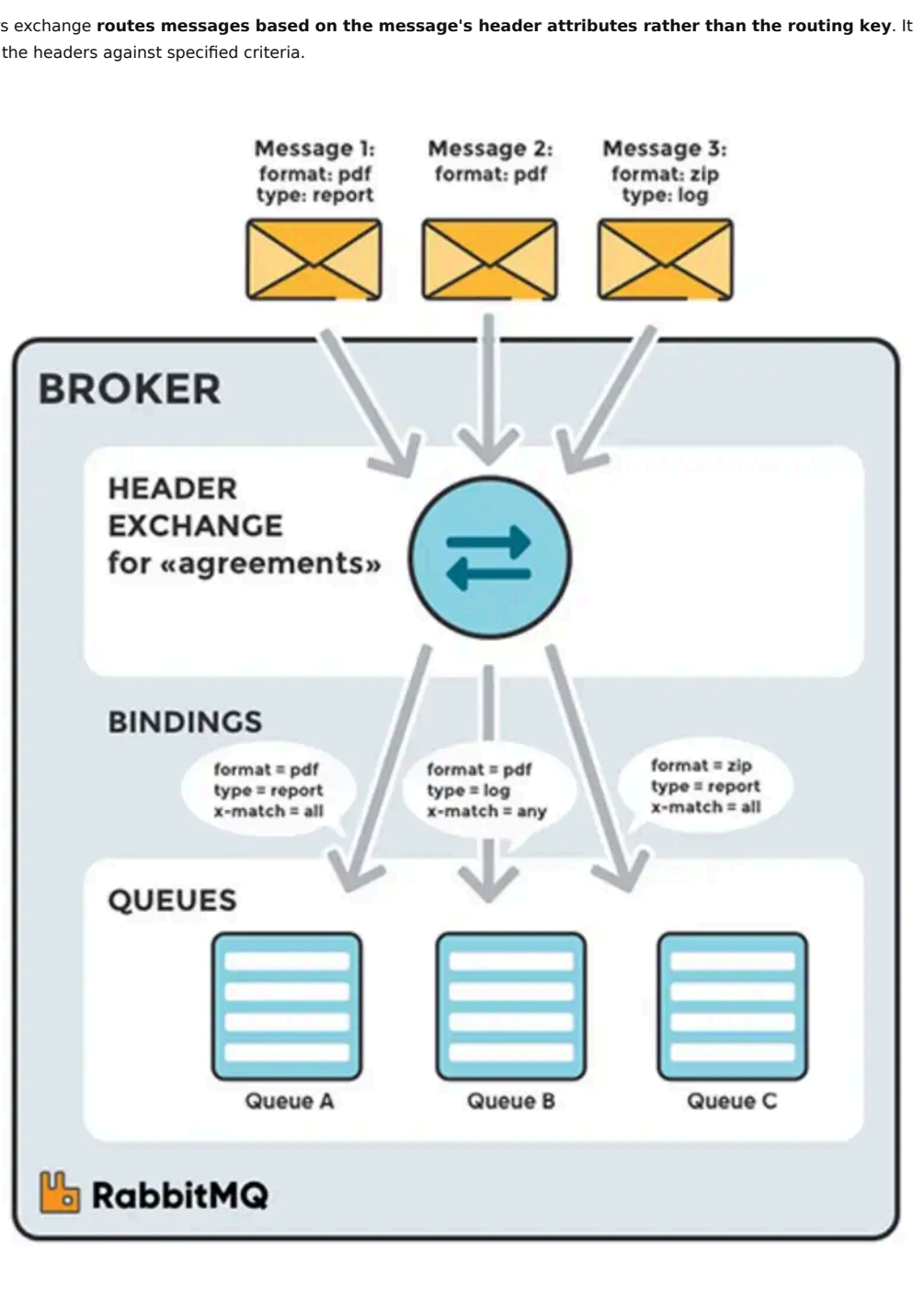
Topic Exchange

A topic exchange **routes messages to one or more queues based on wildcard patterns in the routing key**. This allows for more complex routing logic.


PRODUCER
BROKER
TOPIC EXCHANGE for "agreements"
BINDINGS
A: agreements.eu.berlin.#
B: agreements.#
C: agreements.eu.*.store
QUEUES
Queue A berlin_agreements
Queue B all_agreements
Queue C store_agreements
RabbitMQ

Headers Exchange

A headers exchange **routes messages based on the message's header attributes rather than the routing key**. It matches the headers against specified criteria.


Message 1: format: pdf type: report
Message 2: format: pdf
Message 3: format: zip type: log
BROKER
HEADER EXCHANGE for «agreements»
BINDINGS
format = pdf type = report x-match = all
format = pdf type = log x-match = any
format = zip type = report x-match = all
QUEUES
Queue A
Queue B
Queue C
RabbitMQ

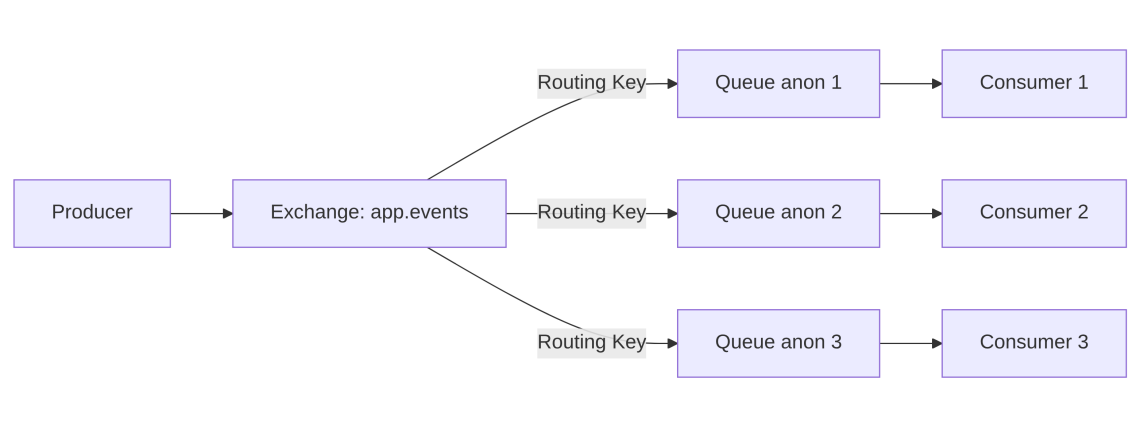
Why `topic` dominates in practice

- Supports a mix of **broadcast** and **selective delivery** in the same system.
- Event-driven architectures, microservices, and notification systems almost always require **selective message delivery**.
- Multiple consumers can subscribe to different patterns without changing producer logic.
- `Direct` is too rigid for complex systems; `fanout` is too blunt for most enterprise needs; `headers` exchanges are rarely used because pattern-matching on routing keys is simpler and more performant.

Communication Patterns

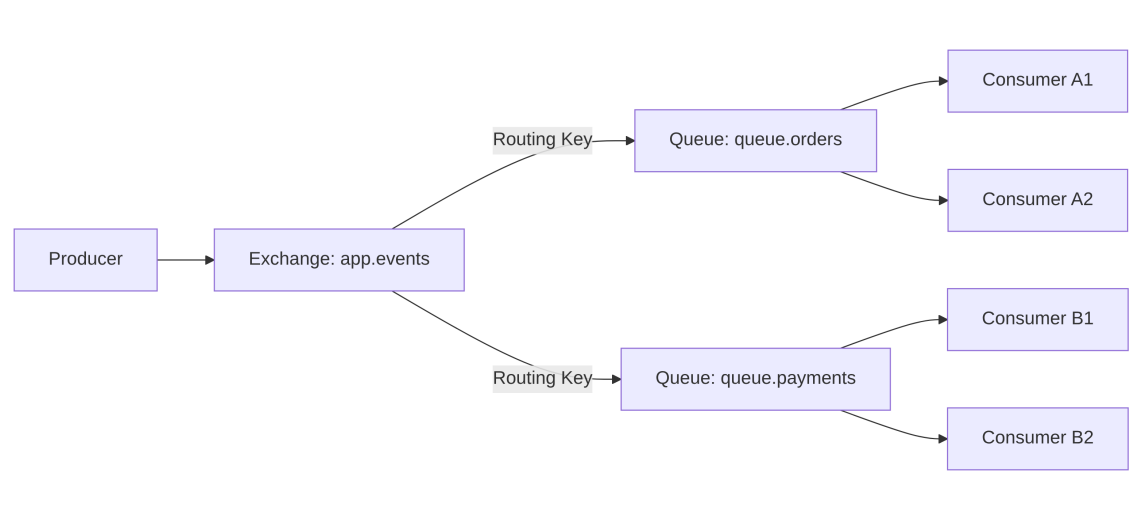
One To Many

Each consumer listens on a dedicated anonymous queue. All queues are connected to the same exchange, ensuring that every message published to the exchange is delivered to all consumers.


Producer
Exchange: app.events
Routing Key
Routing Key
Routing Key
Queue anon 1
Queue anon 2
Queue anon 3
Consumer 1
Consumer 2
Consumer 3

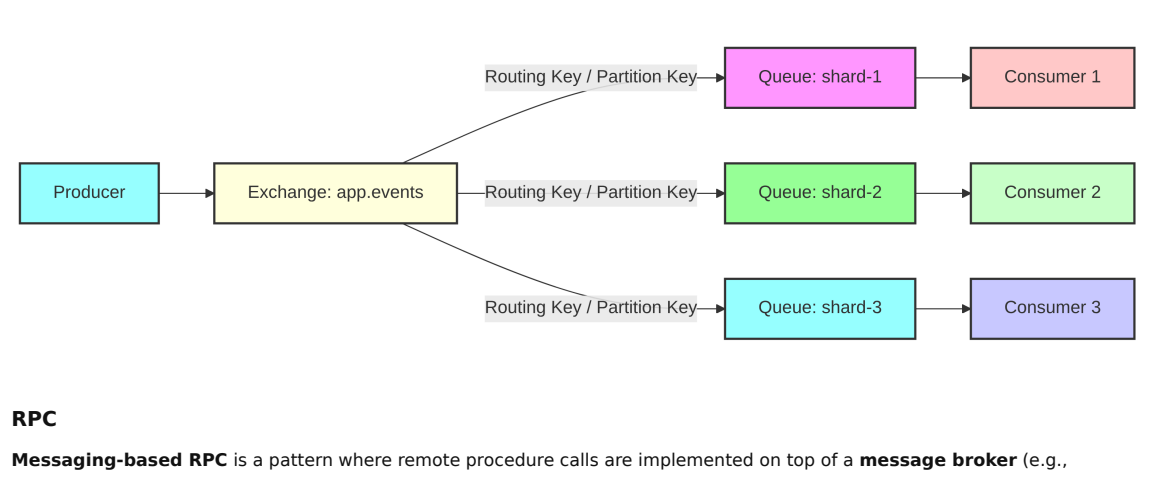
One To Many (Competing Consumers)

All consumers share the same named queue, so each message is delivered to only one consumer. Messages are load-balanced across consumers.


Producer
Exchange: app.events
Routing Key
Routing Key
Queue: queue.orders
Queue: queue.payments
Consumer A1
Consumer A2
Consumer B1
Consumer B2

One To Many (Sharded Consumers)

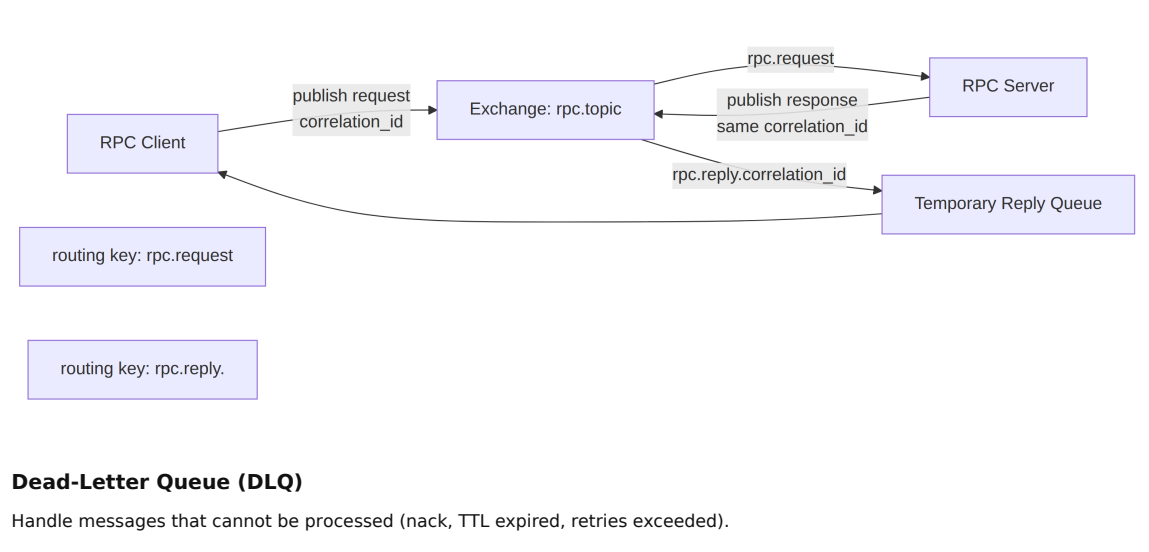
All consumers share the same named queue, so each message is delivered to only one consumer. Messages are load-balanced across consumers. (each consumer receives events related to same entity).


Producer
Exchange: app.events
Routing Key / Partition Key
Routing Key / Partition Key
Routing Key / Partition Key
Queue: shard-1
Queue: shard-2
Queue: shard-3
Consumer 1
Consumer 2
Consumer 3

RPC

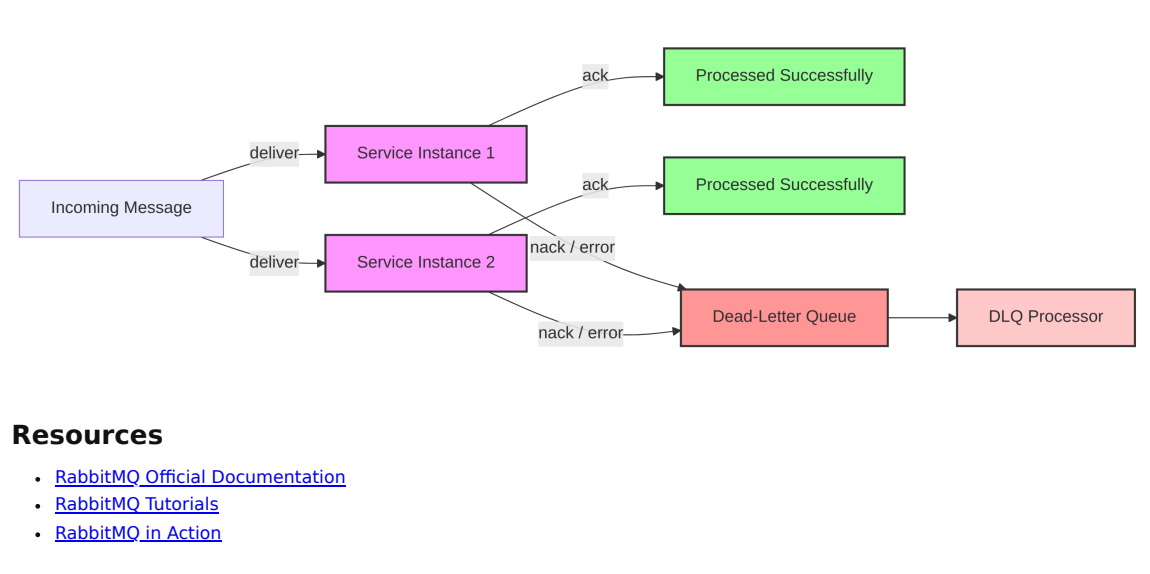
Messaging-based RPC is a pattern where remote procedure calls are implemented on top of a **message broker** (e.g., RabbitMQ), instead of direct network calls.

A client sends a **request message** to a broker, which routes it to a server. The server processes the request and sends back a **response message**. The client typically waits for the reply, matching it using a **correlation identifier**.


RPC Client
publish request correlation_id
Exchange: rpc.topic
rpc.request
RPC Server
publish response same correlation_id
rpc.reply.correlation_id
Temporary Reply Queue
routing key: rpc.request
routing key: rpc.reply.

Dead-Letter Queue (DLQ)

Handle messages that cannot be processed (nack, TTL expired, retries exceeded).


Incoming Message
deliver
deliver
Service Instance 1
Service Instance 2
ack
ack
Processed Successfully
Processed Successfully
nack / error
nack / error
Dead-Letter Queue
DLQ Processor

Resources

- RabbitMQ Official Documentation
- RabbitMQ Tutorials
- RabbitMQ in Action